Modular Gemma 4 Migration Report for CrisisConnect

1. Current repo baseline

The repo already has a good disaster-response base. The README describes CrisisConnect as a unified disaster management platform for citizens, NGOs/field workers, and government agencies. It supports disaster registration, safe-zone routing, NGO resources, SOS-to-responder connection, geofenced alerts, and offline-safe workflows. (GitHub)

The three web portals are already clear:

Portal	Current purpose
Citizen	Map, safe zones, resources, one-tap SOS, public advisories
NGO / field worker	Onboarding, inventory, SOS monitoring, public field updates
Government / admin	Disaster polygons, safe zones, organization approval, alerts, analytics

The repo also has an Android-only Flutter citizen app. It already supports Cognito authentication, AppSync GraphQL, live dashboard/map/resource/news reads, resource requests, SOS creation, and citizen-safe real-time updates. (GitHub)

The current AI layer is useful but not yet Gemma-centered. The existing GraphQL AI operations include citizen guidance, incident briefs, alert drafts, operations recommendations, SOS triage, resource dispatch planning, SOS message preparation, and AI audit review. (GitHub)

The main gap: the current environment and AI Lambda are still wired around **Gemini**, not Gemma. `.env.example` has `GEMINI_API_KEY`, and the AI Lambda references `GEMINI_INTERACTIVE_MODEL`, `GEMINI_ANALYSIS_MODEL`, and the Google Generative Language API. (GitHub)

---

2. Main product change

The new system should be framed as:

When internet is available:

CrisisConnect syncs verified disaster data, safe zones, resources, road/shelter status, alerts, SOPs, and map data.

When internet fails:
The mobile app / web PWA / field command device still uses GPS, cached maps, cached disaster data, and local Gemma 4 to generate safe, grounded, multilingual guidance and operational insights.

This fits the hackathon strongly because the Gemma 4 Good competition focuses on using Gemma 4 for real-world impact, including global resilience, digital equity, safety/trust, post-training, domain adaptation, retrieval, storytelling, public repo, demo, and technical writeup. (AI Competition Hub)

Module A — Replace Gemini with a Gemma 4 AI Platform

A1. Replace direct Gemini calls with provider abstraction

Current problem:

lambda/ai/index.ts
- callGemini()
- GEMINI_API_KEY
- GEMINI_INTERACTIVE_MODEL
- GEMINI_ANALYSIS_MODEL

Change to:

```
lambda/ai/
├── index.ts
├── providers/
│   ├── gemma-provider.ts
│   ├── local-gemma-provider.ts
│   ├── hosted-gemma-provider.ts
│   └── provider-types.ts
├── tasks/
│   ├── citizen-guidance.ts
│   ├── prepare-sos.ts
│   ├── incident-brief.ts
```

```
│   ├── alert-draft.ts
│   ├── operations-recommendation.ts
│   ├── sos-triage.ts
│   └── resource-dispatch.ts
├── safety/
│   ├── guardrails.ts
│   ├── pii.ts
│   ├── prompt-injection.ts
│   └── response-validator.ts
└── grounding/
    ├── context-builder.ts
    ├── source-citations.ts
    └── freshness-check.ts
```

New environment variables:

```
AI_PROVIDER=gemma
GEMMA_RUNTIME=ollama
GEMMA_ENDPOINT=http://localhost:11434
GEMMA_INTERACTIVE_MODEL=gemma4:e4b-it
GEMMA_ANALYSIS_MODEL=gemma4:26b-it
GEMMA_FINETUNED_MODEL=crisisconnect-gemma4-e4b-sos
GEMMA_MODE=local_first
AI_ALLOW_CLOUD_FALLBACK=false
```

For the hackathon, keep the old Gemini provider only as a disabled fallback, or remove it fully to avoid confusing judges.

A2. Add model routing

Use different Gemma modes:

Use case	Recommended model
Citizen mobile/offline guidance	Gemma 4 E2B/E4B quantized
SOS structuring	Fine-tuned Gemma 4 E4B
Sinhala/Tamil/English guidance	Fine-tuned Gemma 4 E4B or 26B
NGO field summaries	Gemma 4 E4B or 26B
Government command briefs	Gemma 4 26B/31B local command-center model

Safety review	Gemma 4 E4B + deterministic rules

Gemma 4 E2B/E4B are specifically positioned for mobile and IoT devices, while 26B/31B are positioned for personal computers and local-first AI servers. (Google DeepMind)

A3. Keep strict JSON schemas

Your existing AI functions already expect structured outputs such as `CitizenGuidance`, `IncidentBrief`, `SosTriage`, and `PreparedSosSubmission`. Keep this design because it is safer than free-form chat. The existing TypeScript types already include metadata such as confidence, source IDs, warnings, human-approval requirement, audit reference, and risk flags. (GitHub)

Add stricter schemas:

```
GemmaResponseMeta {
  modelName: string;
  modelVersion: string;
  adapterVersion?: string;
  runtime: "on_device" | "edge_gateway" | "command_center" | "cloud";
  offlineMode: boolean;
  dataFreshnessMinutes: number;
  groundingSources: string[];
  requiresHumanApproval: boolean;
}
```

Module B — Fine-Tune Gemma 4 for Disaster Behavior

B1. Fine-tuning goal

You are right: this project can use fine-tuning. But the correct purpose is **behavior**, not live facts.

Do not fine-tune Gemma to remember:

- current flood zones
- current shelter capacity
- current NGO inventory
- current SOS cases

- current road closures

Those must come from cached/online data.

Fine-tune Gemma for:

- short emergency instructions
- structured SOS JSON
- urgency classification
- Sinhala/Tamil/English safety wording
- government/NGO response format
- refusal of unsafe advice
- avoiding invented shelters/routes/resources

Google's Gemma tuning docs describe fine-tuning as modifying model behavior for a specific task/domain or role, and they list the general workflow: choose a framework, collect data, tune/test, and deploy. (Google AI for Developers)

B2. Recommended fine-tuning dataset

Create `training/gemma4-disaster-behavior/`.

Dataset files:

```
training/
├── sos_structuring_train.jsonl
├── citizen_guidance_train.jsonl
├── alert_translation_train.jsonl
├── ngo_triage_train.jsonl
├── government_brief_train.jsonl
├── refusal_safety_train.jsonl
└── eval_cases.jsonl
```

Example training pair:

```
{
  "input": {
    "role": "citizen",
    "language": "mixed",
    "message": "water coming inside house grandmother sick 4 people near bridge",
    "context": {
      "disaster_type": "flood",
      "gps_available": true,
      "nearest_safe_zone": "Kolonnawa Community Hall",
      "data_freshness_minutes": 42
```

```
    }
  },
  "output": {
    "incident_type": "flood_trapped",
    "people_count": 4,
    "medical_risk": true,
    "vulnerable_people": ["elderly"],
    "urgency": "high",
    "missing_information": ["exact house number", "water level"],
    "citizen_message": {
      "english": "Your SOS draft is ready. Keep your phone on and move to higher ground only if safe.",
      "sinhala": "...",
      "tamil": "..."
    }
  }
}
```

B3. Fine-tuning method

Use **LoRA/QLoRA**, not full fine-tuning.

Google's docs explicitly list Keras LoRA, Hugging Face Transformers/PEFT, Unsloth, Axolotl, Vertex AI, and other frameworks as Gemma tuning options. They also note that PEFT methods such as LoRA update only a subset of parameters and require fewer compute resources than full tuning. (Google AI for Developers)

Recommended plan:

MVP:
Gemma 4 E4B + prompt/RAG + schema validation

Full hackathon enhancement:
Gemma 4 E4B + LoRA/QLoRA adapter for SOS + multilingual guidance

Advanced:
Gemma 4 26B local command-center model for government/NGO summaries

---

Module C — Offline Data Sync and Local Knowledge Layer

This is the most important feature for proving why Gemma 4 matters.

C1. Add “Emergency Sync Package”

When internet is available, the app should fetch a compact offline package:

```
{
  "generatedAt": "2026-05-17T10:00:00+05:30",
  "validUntil": "2026-05-17T16:00:00+05:30",
  "region": "Colombo",
  "disasters": [],
  "safeZones": [],
  "resources": [],
  "publicAlerts": [],
  "sopDocuments": [],
  "offlineMapTiles": [],
  "riskRules": [],
  "emergencyContacts": []
}
```

Add backend endpoint:

getEmergencySyncPackage(region, userRole, lastSyncTime)

Add local storage:

Platform	Storage
Mobile Flutter	SQLite / Drift / Hive
Citizen web PWA	IndexedDB
NGO web PWA	IndexedDB
Government web	IndexedDB + command-center local server

The repo already has service-worker/PWA positioning and offline-ready experience in the README, so this extends an existing direction rather than adding a random feature. (GitHub)

C2. Add data freshness everywhere

Every Gemma output should show:

Based on data last synced 42 minutes ago.

This is important for trust.

Add to all AI responses:

```
dataFreshness: {
  lastSyncedAt: string;
  ageMinutes: number;
  stale: boolean;
  staleWarning?: string;
}
```

If stale:

“Offline data may be outdated. Use this as guidance only. Follow official instructions when available.”

C3. GPS offline logic

The mobile app already requests Android location permission when needed for safety routing, SOS, or request location capture. (GitHub)

Add:

- offline GPS capture
- last known location
- distance to cached safe zones
- whether user is inside last synced disaster polygon
- offline-safe route explanation

Important design rule:

Deterministic geospatial logic calculates nearest safe zones and risk-zone intersection. Gemma only explains the result in simple language.

Do not let Gemma invent routes.

Module D — Citizen Mobile App Changes

The mobile app should become the strongest Gemma demo because mobile/offline is the clearest reason to use local Gemma.

Current mobile app is Android-only Flutter and already has authentication, live map/resources/news, SOS creation, and real-time updates. (GitHub)

D1. Add local Gemma runtime mode

Add files:

```
citizen_android_app/lib/core/ai/
├── gemma_runtime.dart
├── gemma_prompt_builder.dart
├── gemma_response_parser.dart
├── gemma_safety_validator.dart
└── gemma_model_status.dart
```

Runtime options:

Mode	Use
On-device Gemma 4 E2B/E4B	Best story, true offline
Edge gateway Gemma	Phone connects to local laptop/Jetson/NGO device over hotspot
Cloud fallback disabled	Stronger hackathon story

For demo practicality, you can implement **edge gateway mode first**:

```
Phone offline from internet
↓
Phone connected to local responder hotspot
↓
Local laptop/Jetson runs Gemma 4
↓
Citizen still gets guidance
```

That still proves local Gemma usefulness.

D2. Offline citizen advisor screen

New screen:

features/citizen/offline_advisor/

```
├── offline_advisor_screen.dart
├── emergency_sync_status_card.dart
├── local_gemma_chat_card.dart
├── safe_zone_recommendation_card.dart
└── data_freshness_banner.dart
```

Features:

- “What should I do now?” button
- current GPS
- nearest cached safe zone
- active disaster risk
- cached public alerts
- Gemma-generated short guidance
- Sinhala/Tamil/English toggle

D3. SOS store-and-forward

Add:

```
features/citizen/offline_sos/
├── offline_sos_draft_screen.dart
├── sos_queue_repository.dart
├── sos_sync_worker.dart
└── sos_gemma_structurer.dart
```

Flow:

Citizen writes messy SOS
↓
Gemma structures the SOS locally
↓
App stores signed/queued SOS locally
↓
When internet/SMS/relay returns, it syncs

This is better than pretending SOS works with no communication.

D4. Offline map package

Add:

- cached OpenStreetMap tiles
- cached safe-zone markers

- cached disaster polygon overlays
- cached shelter capacity snapshot

The mobile README already says the map uses OpenStreetMap tiles and backend GeoJSON for affected areas, locations, boundaries, and center points, so this is a natural extension. (GitHub)

D5. Citizen-side Gemma features

Add these mobile features:

1. Offline Situation Advisor
2. SOS Message Structuring
3. Multilingual Safety Guidance
4. Safe-zone Explanation
5. Family Emergency Checklist
6. Stale Data Warning
7. Offline SOS Queue
8. Low-bandwidth SMS fallback draft
9. Voice-friendly short instructions
10. Panic-reducing “what to do next” card

Module E — Citizen Web Portal Changes

Current citizen web portal already has live map, safe zones/resources, one-tap SOS, and public updates. (GitHub)

Add:

src/app/citizen/offline/page.tsx
src/components/citizen/GemmaOfflineAdvisor.tsx
src/components/citizen/SosDraftAssistant.tsx
src/components/citizen/DataFreshnessBanner.tsx
src/lib/offline/emergency-cache.ts
src/lib/offline/indexeddb-store.ts
src/lib/gemma/local-client.ts

Citizen web features:

- “Offline-ready status” panel
- cached shelter list
- cached active warnings

- local Gemma guidance
- “Prepare SOS now, send later” workflow
- multilingual explanation cards
- low-literacy mode with short instructions

Important UI label:

Gemma 4 Local Guidance
Based on cached verified CrisisConnect data.
Last synced: 42 minutes ago.

This makes the Gemma value visible.

Module F — NGO / Field Worker Portal Changes

Current NGO portal already supports onboarding, inventory/resource management, SOS monitoring, accepting SOS incidents, and publishing field updates. (GitHub)

Add:

src/app/ngo/field-copilot/page.tsx
src/components/ngo/GemmaDispatchBrief.tsx
src/components/ngo/SosTriagePanel.tsx
src/components/ngo/ResourcePackingChecklist.tsx
src/components/ngo/OfflineFieldNotes.tsx
src/lib/ngo/offline-field-cache.ts

NGO Gemma modules:

F1. Gemma SOS Triage

Existing operation: `triageSosCase`.

Upgrade it to:

- severity explanation
- urgency classification
- nearest responder suggestion
- medical/vulnerable-person flags
- missing-information questions

- human approval required

The current GraphQL mutation already returns `severity`, `urgency`, `responderIds`, rationale, recommendations, metadata, warnings, and risk flags, so the structure is already close. (GitHub)

F2. Gemma Field Dispatch Brief

For each responder:

“You are assigned to 3 cases. Case 1 has elderly medical risk. Case 2 needs water and dry food. Carry first-aid kit, water packs, torch, rope, and phone power bank.”

F3. Gemma Resource Packing Checklist

Input:

SOS case + resource inventory + disaster type + distance + shelter capacity

Output:

- supplies to carry
- what to confirm before leaving
- what to report after arrival

F4. Offline NGO notes

Field worker records:

“family evacuated, one elderly patient, needs medicine”

Gemma converts to:

```
{
  "case_status": "evacuated",
  "medical_followup": true,
  "resources_needed": ["medicine"],
  "public_update_safe": false,
  "government_summary": "One elderly evacuee requires medical follow-up."
}
```

Module G — Government / Admin Portal Changes

Current government/admin portal supports disaster polygons, severity, safe zones, occupancy, organization approval, SMS/email/push alerts, finance/resource analytics, and dashboards. (GitHub)

Add:

src/app/admin/gemma-command-center/page.tsx
src/components/admin/GemmaIncidentBrief.tsx
src/components/admin/GemmaAlertComposer.tsx
src/components/admin/GemmaRiskReview.tsx
src/components/admin/EmergencySyncPackagePublisher.tsx
src/components/admin/ModelAuditDashboard.tsx
src/components/admin/DataFreshnessMap.tsx

Government Gemma modules:

G1. Gemma Incident Brief

Existing operation: `generateIncidentBrief`.

Upgrade it to include:

- incident summary
- high-risk clusters
- shelter pressure
- resource gaps
- recommended actions
- assumptions
- stale/missing data warnings

The current GraphQL mutation already returns headline, summary, translations, rationale, recommendations, metadata, audit, and risk flags. (GitHub)

G2. Gemma Operations Recommendation

Existing operation: `recommendOperations`.

Upgrade it to:

- next 1 hour / 6 hours / 24 hours action plan

- responder deployment priorities
- resource rebalancing
- shelter overflow warnings
- unresolved SOS clustering

The existing operation already returns timeframe, rationale, recommendations, related IDs, metadata, warnings, and risk flags. (GitHub)

G3. Gemma Alert Composer

Existing operation: `generateAlertDraft`.

Upgrade it to:

- English/Sinhala/Tamil alert draft
- panic-risk checker
- missing details detector
- geofenced target explanation
- approval workflow
- SMS-length version
- push-notification version
- public-feed version

The current alert draft mutation already returns title, channels, English/Sinhala/Tamil content, rationale, metadata, warnings, and risk flags. (GitHub)

G4. Emergency Sync Package Publisher

This is a new government feature.

Government clicks:

Publish Offline Emergency Package

System creates:

- current disaster polygons
- safe-zone capacity snapshot
- active alerts
- official SOPs
- emergency contacts
- map tile region manifest
- risk rules

Then citizen/NGO apps download it before connectivity fails.

This is one of the best ways to show practical disaster resilience.

Module H — Database and Backend Changes

The current database already has disasters, safe zones, resources, resource requests, SOS signals, notifications, news updates, financial aid, and AI audit logs. (GitHub)

Add a new migration:

db/migrations/002_gemma_offline_intelligence.sql

Suggested tables:

```
CREATE TABLE emergency_sync_packages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  region TEXT NOT NULL,
  disaster_id UUID REFERENCES disasters(id),
  package_json JSONB NOT NULL,
  generated_by TEXT REFERENCES profiles(id),
  generated_at TIMESTAMPTZ DEFAULT now(),
  valid_until TIMESTAMPTZ,
  checksum TEXT,
  version INTEGER DEFAULT 1
);

CREATE TABLE device_sync_status (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id TEXT REFERENCES profiles(id),
  device_id TEXT NOT NULL,
  last_sync_at TIMESTAMPTZ,
  package_id UUID REFERENCES emergency_sync_packages(id),
  offline_capable BOOLEAN DEFAULT false
);

CREATE TABLE offline_sos_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  local_id TEXT NOT NULL,
  sender_id TEXT REFERENCES profiles(id),
  payload JSONB NOT NULL,
  status TEXT DEFAULT 'queued',
```

```
  created_offline_at TIMESTAMPTZ,
  synced_at TIMESTAMPTZ
);

CREATE TABLE gemma_model_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  action TEXT NOT NULL,
  model_name TEXT NOT NULL,
  model_version TEXT,
  adapter_version TEXT,
  runtime TEXT,
  offline_mode BOOLEAN,
  data_freshness_minutes INTEGER,
  input_hash TEXT,
  output_hash TEXT,
  status TEXT,
  confidence NUMERIC,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE gemma_eval_results (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  model_name TEXT NOT NULL,
  adapter_version TEXT,
  eval_suite TEXT NOT NULL,
  json_valid_rate NUMERIC,
  safety_pass_rate NUMERIC,
  hallucination_rate NUMERIC,
  multilingual_score NUMERIC,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Also extend `ai_audit_logs` or link it to `gemma_model_runs`. The current audit table already stores action, role, user, model, status, review status, confidence, source IDs, warnings, blocking flags, PII detection, prompt-injection risk, unsafe content, latency, token usage, and timestamps. (GitHub)

Module I — GraphQL/API Changes

Keep existing AI operations but rename them in product wording from generic AI to Gemma.

Current operations to keep and upgrade:

getCitizenGuidance

prepareSosSubmission
generateIncidentBrief
generateAlertDraft
recommendOperations
triageSosCase
recommendResourceDispatch
getAiAuditLogs
reviewAiAuditLog

Add new operations:

query GetEmergencySyncPackage($region: String!, $lastSyncAt: String)
mutation PublishEmergencySyncPackage($input: EmergencySyncPackageInput!)
mutation QueueOfflineSos($input: OfflineSosInput!)
mutation SyncOfflineSosQueue($items: [OfflineSosInput!]!)
query GetGemmaModelStatus
query GetGemmaAuditDashboard($limit: Int)
mutation EvaluateGemmaOutput($input: GemmaEvalInput!)
mutation GenerateOfflineCitizenInsight($input: OfflineInsightInput!)

Module J — Safety and Trust Layer

This is critical because disaster AI can be dangerous.

Rules:

1. Gemma cannot invent shelters, roads, resource stock, or responders.
2. Geospatial calculations must be deterministic.
3. All high-impact actions require human approval.
4. All outputs show data freshness.
5. SOS and dispatch suggestions include rationale.
6. Alerts go through government approval.
7. Personal data is minimized in summaries.
8. Unsafe route/medical certainty is blocked.

Your repo already has a strong foundation because current AI responses include audit metadata, risk flags, source IDs, warnings, and human approval fields. (GitHub)

Add visible labels:

Gemma output type:
- Advisory
- Requires approval

- Blocked
- Stale data warning
- Offline mode

Module K — Documentation and Hackathon Story Changes

Add these files:

docs/gemma-4-architecture.md
docs/offline-first-design.md
docs/fine-tuning-plan.md
docs/model-card-crisisconnect-gemma.md
docs/safety-and-human-approval.md
docs/demo-script-gemma4.md
docs/kaggle-writeup.md
docs/local-gemma-setup.md

Update README title:

CrisisConnect Edge — Gemma 4-Powered Offline Disaster Intelligence

Add a clear section:

Why Gemma 4?
- Runs locally during weak/no internet
- Keeps sensitive SOS/disaster data on device or local command hardware
- Supports multilingual emergency guidance
- Can be fine-tuned for structured SOS and official alert style
- Works with function-calling/tool-style workflows
- Enables edge AI for citizens, NGOs, and command centers

Gemma 4's official pages support this positioning because they describe open weights, tuning/deployment, edge/mobile/browser model sizes, multilingual support, function calling, and offline edge processing. (Google AI for Developers)

Module L — Suggested Demo Flow

Use this for the hackathon video:

Scene 1 — Internet available

Government creates a Colombo flood polygon. The existing demo flow already starts with government creating a flood disaster polygon, sending a geofenced warning, citizen seeing the map/safe zone, citizen submitting SOS, NGO accepting dispatch, and government dashboard updating. (GitHub)

Add:

Government publishes Emergency Sync Package.
Citizen and NGO apps download it.
Gemma model status shows: Ready for offline guidance.

Scene 2 — Internet fails

Show:

Network: offline
GPS: available
Emergency package: last synced 38 minutes ago
Gemma 4: running locally

Citizen asks:

“Water is coming into my house. My grandmother is sick. What should I do?”

Gemma returns:

- short safety guidance
- nearest cached safe zone
- warning about stale data
- Sinhala/Tamil/English option

Scene 3 — SOS store-and-forward

Citizen types messy SOS.

Gemma structures it:

```
{
  "incidentType": "flood_trapped",
```

```
  "peopleCount": 4,
  "medicalRisk": true,
  "urgency": "high",
  "missingInfo": ["water level", "exact access route"]
}
```

App queues it locally.

Scene 4 — Network returns / NGO gateway appears

Queued SOS syncs.

NGO portal receives it.

Gemma gives field dispatch brief:

High priority. Elderly medical risk. Carry first aid, drinking water, dry food, torch, and stretcher if available.

Scene 5 — Government command center

Gemma summarizes:

3 high-risk SOS reports near Kolonnawa.
Shelter A is 86% full.
Recommend opening overflow shelter and dispatching rescue team.

Government reviews and approves the alert.

Highest-priority implementation order

Do it in this order:

Priority	Change	Why
1	Replace Gemini env/code with Gemma provider	Required for hackathon credibility
2	Add local Gemma runtime/proxy	Proves “local LLM” value
3	Add emergency sync package	Makes offline intelligence grounded

4	Add mobile offline advisor	Best user-facing Gemma demo
5	Add SOS store-and-forward	Realistic offline behavior
6	Add fine-tuned SOS/alert adapter	Shows post-training/domain adaptation
7	Add government sync publisher	Makes system-level story stronger
8	Add Gemma audit/model dashboard	Shows safety and trust
9	Update README/writeup/video	Makes judges understand why Gemma matters

---

Final recommendation

Your final project should not be described as:

> “A disaster app with an AI chatbot.”

It should be described as:

> **A local-first disaster intelligence system where Gemma 4 runs on citizen devices, field responder devices, or local command-center hardware to provide grounded guidance, structured SOS preparation, multilingual alerts, and operational insights when cloud AI and internet connectivity are unreliable.**

That is the strongest way to show the importance of Gemma 4 in this repo.